

Formalizing Lagrange’s Theorem in Lean 4 / Mathlib

A Practical Plan

Expert guide: Lean 4, Mathlib, and formalized group theory

Contents

1	Overview and Goal	2
2	Main Mathematical Lemmas Required	2
3	Relevant Mathlib Definitions and Theorems	2
4	Sensible Implementation Order	3
5	Likely Difficult Lean-Specific Parts	4
5.1	Instance management	4
5.2	Cardinality of the quotient type	4
5.3	Sigma-type versus Finset-based counting	4
5.4	Connecting Set and Fintype cardinalities	4
5.5	Rewriting under binders	4
5.6	The commutativity mismatch	4
6	Minimal Proof Skeleton	5
7	Key Mathlib Lemma Signatures for Quick Reference	6
8	Summary and Recommendations	7
8.1	If your goal is a working proof	7
8.2	If your goal is a pedagogical / from-scratch proof	7
8.3	Cardinality formulation	7

1 Overview and Goal

Theorem 1.1 (Lagrange). *Let G be a finite group and $H \leq G$ a subgroup. Then*

$$|H| \mid |G|.$$

Equivalently, $|G| = [G : H] \cdot |H|$, where $[G : H]$ is the index of H in G (the number of left cosets of H in G).

Strategy. Mathlib already contains the full proof of Lagrange’s theorem (`Subgroup.card_subgroup_dvd_card`) together with most of the required infrastructure. The plan below first assembles the mathematical landscape, then shows how to reach the theorem with minimal boilerplate by leaning heavily on existing Mathlib results.

The canonical Lean statement we target is:

```
theorem lagrange (G : Type*) [Group G] [Fintype G]
  (H : Subgroup G) :
    Fintype.card H      Fintype.card G := by
  exact Subgroup.card_subgroup_dvd_card H
```

If this one-liner is not pedagogically satisfying, the remainder of this document details how to build the proof from first principles, following the proof in Mathlib closely.

2 Main Mathematical Lemmas Required

The classical proof has four logical steps:

- L1. Left cosets partition G .** Define the left coset of $g \in G$ with respect to H as $gH = \{gh \mid h \in H\}$. The relation $a \sim b \iff a^{-1}b \in H$ is an equivalence relation on G , and the equivalence classes are exactly the left cosets.
- L2. Each coset has the same cardinality as H .** The map $\phi_g: H \rightarrow gH, h \mapsto gh$ is a bijection, so $|gH| = |H|$ for all $g \in G$.
- L3. Cardinality of a disjoint union.** If a finite set S is partitioned into k classes each of size m , then $|S| = k \cdot m$.
- L4. Divisibility conclusion.** Combining L2 and L3 gives $|G| = [G : H] \cdot |H|$, which implies $|H| \mid |G|$.

3 Relevant Mathlib Definitions and Theorems

Table 1 lists the key Mathlib identifiers. All live under `Mathlib.GroupTheory.*` or `Mathlib.Data.Fintype.*` unless noted.

Search tips.

- Use `exact?`, `apply?`, and `simp?` liberally inside tactic blocks to find lemma names automatically.
- Search <https://loogle.lean-lang.org> with patterns like `(Subgroup ?H) → Nat.dvd` or `card_subgroup`.
- The `#check Subgroup.card_subgroup_dvd_card` command immediately reveals the exact signature in your editor.

Mathlib identifier	Purpose
<code>Subgroup G</code>	The type of subgroups of G ; carries <code>[Group G]</code>
<code>QuotientGroup.quotient H</code>	The type G/H (left cosets)
<code>leftCoset g H</code>	The set gH as a term of type <code>Set G</code>
<code>QuotientGroup.leftRel H</code>	The equivalence relation $a \sim b \iff a^{-1}b \in H$
<code>Subgroup.leftCosetEquivSubgroup</code>	Bijection $H \simeq gH$ (the map $h \mapsto gh$)
<code>Fintype.card_congr</code>	Transfers cardinality along an <code>Equiv</code>
<code>Fintype.card_sigma</code>	$ \sum i, f i = \sum_i f i $
<code>Finpartition.card_parts_eq</code>	Cardinality from a <code>Finpartition</code>
<code>Subgroup.card_subgroup_dvd_card</code>	The main theorem
<code>Subgroup.index</code>	Index $[G : H]$ as a natural number
<code>Subgroup.card_mul_index</code>	$ H \cdot [G : H] = G $
<code>Nat.dvd_of_mul_dvd_mul_left</code>	Arithmetic: $km = n \Rightarrow k \mid n$
<code>Fintype.ofFinset</code>	Builds <code>Fintype</code> instances from <code>Finset</code>
<code>Quotient.fintype</code>	<code>Fintype</code> instance for G/H when G is finite
<code>Set.toFinset_card</code>	Connects <code>Set</code> cardinality with <code>Fintype.card</code>

Table 1: Core Mathlib identifiers for the Lagrange proof.

4 Sensible Implementation Order

Follow this sequence, confirming each step compiles before proceeding.

- Step 1. Set up imports and variables.** Import the relevant Mathlib modules and declare the universe polymorphic variables. This surfaces any missing instances early.
- Step 2. State the equivalence relation lemma (L1).** Show that `QuotientGroup.leftRel H` is an equivalence relation and that its classes are left cosets. (In practice, just cite `Setoid.iseqv`; Mathlib already proves this.)
- Step 3. Prove the coset-size bijection (L2).** Exhibit the `Equiv` between `H` and `leftCoset g H` and transport the `Fintype.card` equality.
- Step 4. Express G as a disjoint union of cosets (L3).** Use the `Finpartition` API or the sigma-type decomposition $G \simeq \sum (c : G/H), c$ and count via `Fintype.card_sigma`.
- Step 5. Derive the index equation (L4).** Conclude $|G| = [G : H] \cdot |H|$ using `Subgroup.card_mul_index`.
- Step 6. Extract divisibility.** From $|G| = [G : H] \cdot |H|$, apply `Dvd.intro` (or `<Subgroup.index H, ...>`) to close the goal $|H| \mid |G|$.

5 Likely Difficult Lean-Specific Parts

5.1 Instance management

Lean’s typeclass system requires that both `Fintype G` and `Fintype H` are available. Because H is a `Subgroup G`, the instance `Subgroup.fintype` is synthesised automatically when `Fintype G` is present—but you may need to provide it explicitly in intermediate lemmas using `haveI : Fintype H := inferInstance`.

```
-- Make Fintype H explicit if synthesis fails
haveI : Fintype H := Subgroup.fintype H
```

5.2 Cardinality of the quotient type

`Quotient (QuotientGroup.leftRel H)` is the type of left cosets. Proving it is `Fintype` requires a `DecidableEq` instance on G , which in turn requires decidable membership in H . This is available for `Fintype G` automatically, but you may need:

```
haveI : DecidableEq G := Classical.decEq G
haveI : Fintype (G / H) := inferInstance -- synthesised from
Fintype G
```

5.3 Sigma-type versus Finset-based counting

Two styles of counting appear in Mathlib:

- **Sigma decomposition:** $G \simeq \Sigma (c : G/H), c$, proved via `Equiv.sigmaFibEquiv` combined with `QuotientGroup.mk_surjective`. Then apply `Fintype.card_sigma` and the constant-fibre lemma.
- **Finpartition sum:** build a `Finpartition` of `Finset.univ` whose parts are `leftCoset g H |>.toFinset`, then sum their cardinalities.

The sigma approach is cleaner in Lean 4 / Mathlib 4 because the `Fintype.card_sigma` equation is already proved. Prefer it.

5.4 Connecting Set and Fintype cardinalities

`leftCoset` returns a `Set G`, but `Fintype.card` operates on types (not sets). Bridge this with:

```
-- card of the subtype {x // x ∈ s} equals s.toFinset.card
rw [Set.toFinset_card]
```

or go via the `Subtype` built from the coset.

5.5 Rewriting under binders

When summing over `Fintype.card_sigma`, you need to rewrite inside the sum $\sum_{c:G/H} |c| = \sum_{c:G/H} |H|$. Use `Fintype.sum_congr` and supply the per-fibre equivalence as a function.

5.6 The commutativity mismatch

`Subgroup.card_mul_index` states $|H| \cdot [G : H] = |G|$, whereas the sigma counting gives $[G : H] \cdot |H| = |G|$. Close the gap with `Nat.mul_comm` or `mul_comm`.

6 Minimal Proof Skeleton

The skeleton below mirrors the six-step plan from Section 4. Proof obligations are marked `sorry` and annotated with the lemma or tactic expected to close them.

```
-- =====
-- Lagrange's Theorem: |H|      |G|
-- Lean 4 / Mathlib 4 skeleton
-- =====

import Mathlib.GroupTheory.Coset
import Mathlib.GroupTheory.Index
import Mathlib.Data.Fintype.Card

open Fintype Subgroup QuotientGroup

variable {G : Type*} [Group G] [Fintype G] [DecidableEq G]

--

-- Step 1 (helper): each coset gH is in bijection with H
--

/-- The left-multiplication map h      g * h is a bijection H      gH.
-/
noncomputable def cosetEquiv (H : Subgroup G) (g : G) :
  H      leftCoset g (H : Set G) where
  toFun      := fun h , hH => g * h, mem_leftCoset g h H
  invFun      := fun x , h x =>
    g      * x, by
      obtain h , hH, rfl := hx
      simp [H.mul_mem (H.inv_mem (H.one_mem)) hH]
      -- sorry      fill with: exact H.inv_mem_of_mem ...
  left_inv      := by intro h , _ ; simp
  right_inv      := by intro x , _ ; simp [mul_assoc]

--

-- Step 2: card of each coset equals card of H
--

lemma card_leftCoset_eq (H : Subgroup G) (g : G) :
  Nat.card (leftCoset g (H : Set G)) = Nat.card H := by
  exact Nat.card_congr (cosetEquiv H g)
  -- alternatively: exact Fintype.card_congr (cosetEquiv H g)

--

-- Step 3: sigma decomposition G      (c : G      H), c
--

/-- G is equivalent (as a type) to the sigma of its left cosets. -/
noncomputable def sigmaCosetEquiv (H : Subgroup G) :
  G      (c : G      H), c where
  toFun      := fun g => mk g, g, rfl
  -- The coset containing g, together with g as its witness
  invFun      := fun _ , g, _ => g
  left_inv      := by intro g; rfl
  right_inv      := by
    intro c , g, h g
```

```

    simp only [Sigma.mk.inj_iff]
    exact hg .symm, by subst hg; rfl
    -- sorry      handle the dependent equality carefully

--

-- Step 4: count via the sigma decomposition
--

lemma card_eq_index_mul_card (H : Subgroup G) :
  Fintype.card G = H.index * Fintype.card H := by
  -- Use the pre-proved Mathlib lemma directly
  exact (H.card_mul_index).symm
  -- If building from scratch:
  --   rw [      Fintype.card_sigma]
  --   apply Fintype.card_congr (sigmaCosetEquiv H)
  --   ... then rewrite each fibre card using card_leftCoset_eq

--

-- Step 5: divisibility
--

lemma card_subgroup_dvd_card (H : Subgroup G) :
  Fintype.card H      Fintype.card G := by
  rw [card_eq_index_mul_card H]
  exact Dvd.intro_left H.index rfl
  -- i.e.,   H .index, by ring

--

-- Final theorem (also available as Subgroup.card_subgroup_dvd_card)
--

theorem lagrange (H : Subgroup G) :
  Fintype.card H      Fintype.card G :=
  card_subgroup_dvd_card H
  -- one-liner alternative: Subgroup.card_subgroup_dvd_card H

```

7 Key Mathlib Lemma Signatures for Quick Reference

```

-- The index of H in G (number of left cosets)
#check Subgroup.index          -- Subgroup G

-- |H| * [G : H] = |G| (the fundamental equation)
#check Subgroup.card_mul_index
-- Subgroup.card_mul_index :      {G : Type u_1} [inst : Group G]
--   [inst_1 : Fintype G] (H : Subgroup G),
--   Fintype.card H      * H.index = Fintype.card G

-- Direct divisibility (the theorem itself)
#check Subgroup.card_subgroup_dvd_card
-- Subgroup.card_subgroup_dvd_card :      {G : Type u_1}
--   [inst : Group G] [inst_1 : Fintype G] (H : Subgroup G),
--   Fintype.card H      Fintype.card G

-- Bijection between coset and subgroup

```

```
#check Subgroup.leftCosetEquivSubgroup
-- leftCosetEquivSubgroup : {G : Type u_1} [inst : Group G]
--   (H : Subgroup G) (g : G), H (leftCoset g H)

-- Sigma card splitting
#check Fintype.card_sigma
-- Fintype.card_sigma : { : Type u_1} { : Type u_2}
--   [inst : Fintype ] [inst_1 : (a : ), Fintype ( a)],
--   Fintype.card ( a, a) = a, Fintype.card ( a)
```

8 Summary and Recommendations

8.1 If your goal is a working proof

```
import Mathlib.GroupTheory.Index

theorem lagrange {G : Type*} [Group G] [Fintype G]
  (H : Subgroup G) : Fintype.card H Fintype.card G :=
  Subgroup.card_subgroup_dvd_card H
```

This compiles immediately. The entire mathematical content is encapsulated in Mathlib.

8.2 If your goal is a pedagogical / from-scratch proof

Follow the six-step plan (Section 4) and fill in the `sorry`s in the skeleton (Section 6) in order. The hardest steps are the sigma-type dependent equality in `sigmaCosetEquiv` and managing `Fintype` instances for cosets-as-subtypes. Use `Classical.decEq` and `haveI : Fintype ... := inferInstance` liberally, and lean on `exact?` / `apply?` to identify the right Mathlib lemma at each gap.

8.3 Cardinality formulation

The index equation $|G| = [G : H] \cdot |H|$ (a stronger statement than pure divisibility) is `Subgroup.card_mul_index`. Both formulations are in Mathlib; the divisibility version follows from the multiplication version by `Dvd.intro_left`.

Golden rule. Before proving any group-theory lemma from scratch in Lean 4, run `exact?` or search `LOOGLE`. Mathlib’s `GroupTheory/` hierarchy is remarkably complete, and most standard results about cosets, indices, and finite group cardinalities are already there.

Mathlib version assumed: v4.x (post Lean 4 port, 2024–2026). Identifiers may shift across Mathlib releases; use `#check` and `exact?` to resolve any discrepancies.